Project 4

Finishing the Compiler

- Current Compiler:
  - Source code -> Tokens
  - Tokens -> AST
  - AST -> Typechecked AST

Finishing the Compiler

- Current Compiler:
  - Source code -> Tokens
  - Tokens -> AST
  - AST -> Typechecked AST
- Complete Compiler:
  - Typechecked AST -> IR

Finishing the Compiler

- Current Compiler:
  - Source code -> Tokens
  - Tokens -> AST
  - AST -> Typechecked AST
- Complete Compiler:
  - Typechecked AST -> IR // Lowering

Finishing the Compiler

- Current Compiler:
  - Source code -> Tokens
  - Tokens -> AST
  - AST -> Typechecked AST
- Complete Compiler:
  - Typechecked AST -> IR // Lowering
  - IR -> Working Binary

Finishing the Compiler

- Current Compiler:
  - Source code -> Tokens
  - Tokens -> AST
  - AST -> Typechecked AST
- Complete Compiler:
  - Typechecked AST -> IR // Lowering
  - IR -> Working Binary // CodeGen

Project 4

- Lowering and Codegen
- YOU will implement Lowering
- And partially implement Codegen

GOTO - The IR

- GOTO is a simple IR language with 17 instructions
- CodeGen is fully implemented for 14 instructions
  - WriteToFile, ReadFromFile, Input - Need to be implemented by you
- The codegen takes a GOTO program and produces a C program
- To get a working binary, compile the C program

GOTO Statements

Complete C Statements

```
329 /**
330  * Assignment.
331  *
332  * Example emitted C:
333  * x = 5;
334  */
335 class Assign extends IRStmt {
336     public final Var target;
337     public final IRExpr value;
338
```

```
/**
 * Array allocation.
 *
 * Example emitted C:
 * arr = realloc(&arr, size * sizeof({this.array.type}));
 */
class ArrayAlloc extends IRStmt {
    public final Var array;
    public final IRExpr size;
```

```
401 /**
402  * If statement implemented with goto statements
403  *
404  * Example emitted C:
405  * if (x < 5) goto LABEL_TRUE;
406  * goto LABEL_FALSE;
407  */
408 class IfStmt extends IRStmt {
409     public final IRExpr cond;
410     public final String trueLabel;
411     public final String falseLabel;
```

```
428 /**
429  * Goto statement.
430  *
431  * Example emitted C:
432  * goto LABEL;
433  */
434 class Goto extends IRStmt {
435     public final String label;
436
```

```
447 /**
448  * Label definition.
449  *
450  * Example emitted C:
451  * LABEL:
452  */
453 class Label extends IRStmt {
454     public final String name;
455
```

```
466 /**
467  * Return statement.
468  *
469  * Example emitted C:
470  * return x;
471  */
472 class ReturnStmt extends IRStmt {
473     public final IRExpr value;
```

GOTO Expressions

Building Blocks

```
202 class Var extends IRExpr {
203     public final String name;
```

```
216 /**
217  * Literal constant.
218  *
219  * Represents either a hardcoded int or string.
220  */
221 class Literal extends IRExpr {
222     public final Object value;
223
```

```
235 /**
236  * Binary operation.
237  *
238  * Represents operations like +, *, <, etc.
239  *
240  * Example emitted C:
241  * y + z
242  * x < 5
243  */
244 class BinOp extends IRExpr {
245     public final String op;
246     public final IRExpr left, right;
247
```

```
261 /**
262  * Unary operation.
263  *
264  * Example emitted C:
265  * +y
266  * *p
267  */
268 class UnaryOp extends IRExpr {
269     public final String op;
270     public final IRExpr expr;
271
```

```
/**
 * Function call.
 *
 * Example emitted C:
 * f()

 * Functions do not need arguments, because all
 * variables are global in the final C file.
 */
class Call extends IRExpr {
    public final String func;
```

```
/**
 * Array load: reading from an array.
 *
 * Example emitted C:
 * arr[i]
 */
class ArrayLoad extends IRExpr {
    public final Var array;
    public final IRExpr index;
```

GOTO Programs

- Ir.java contains the Program class

```java
class Program {
    private int unique_name_counter;
    private int unique_label_counter;

    public ArrayList<Var> globals;
    public ArrayList<Function> funcs;

    public Program() {
        this.unique_name_counter = 0;
        this.unique_label_counter = 0;
        this.globals = new ArrayList<>();
        this.funcs = new ArrayList<>();
    }

    public String getUniqueVarName() {
        return "_x"+(++this.unique_name_counter);
    }

    public String getUniqueLabelName() {
        return "LABEL"+(++this.unique_label_counter);
    }
}
```

GOTO Programs

- Ir.java contains the Program class
- A GOTO program contains a list of functions, and a list of vars

```java
class Program {
    private int unique_name_counter;
    private int unique_label_counter;

    public ArrayList<Var> globals;
    public ArrayList<Function> funcs;

    public Program() {
        this.unique_name_counter = 0;
        this.unique_label_counter = 0;
        this.globals = new ArrayList<>();
        this.funcs = new ArrayList<>();
    }

    public String getUniqueVarName() {
        return "_x"+(++this.unique_name_counter);
    }

    public String getUniqueLabelName() {
        return "LABEL"+(++this.unique_label_counter);
    }
}
```

Vars

- ALL
- ALL
- **<u>ALL!</u>**
- Variables in GOTO are global
    - No exceptions
    - Every Variable
    - ….**<u>ALL!</u>**

```c
#include <stdlib.h>
#include <stdio.h>

int x;
int y;

int func() { return y; }
int main() { x = func(); }
```

```java
class Function {
    public ArrayList<GOTO> instr;
    public String name;
    public String returntype;

    public Function(String name, String ret) {
        this.name = name;
        this.instr = new ArrayList<>();
        this.returntype = ret;
    }
}
```

Functions

- NO
- NO
- **NO!**
- Function’s have parameters
  - No exceptions
  - Every Function
  - ….**NO!**

```c
#include <stdlib.h>
#include <stdio.h>

int x;
int y;

int func() { return y; }
int main() { x = func(); }
```

Presents alot of Problems

- Geaux programs can have two variables named x!

```
1 fun int func1() {
2     var int x = 1;
3 }
4 fun int func2() {
5     var int x = 2;
6 }
```

```c
#include <stdlib.h>
#include <stdio.h>

int x;
int x;

int func1() { x = 1; }
int func2() { x = 2; }

```

Presents alot of Problems

- Geaux programs can have two variables named x!
  - The solution is to rename the variables to unique names
  - Anytime 1 variable shadows another, a renaming must take place
  - Easiest way: Use the Scope from project 3

Presents alot of Problems

- Geaux programs can have two variables named x!
  - The solution is to rename the variables to unique names
  - Anytime 1 variable shadows another, a renaming must take place
  - Easiest way: Use the Scope from project 3
- Geaux programs *do* have function parameters
  - You will need to convert to GOTO functions with no parameters.

What YOU have to do

1. Write new passes (just like project 3) that take AST nodes and generate GOTO statements
2. Deal with variable renaming and Function parameters
3. Finish implementing ReadFromFile, WriteToFile, and Input
    a. Check the comments for details
    b. Don’t forget to write the codegen for these too

Two More Things You Should Know

- GOTO does not include a “loop”
  - You will need to figure out a way to get GOTO to loop
- Geaux program arrays are really dynamic lists
  - If an array is accessed
    - Arr[i] = x;
    - If i > sizeOfArr => Arr must be resized to be bigger!
  - Your goto should include checks to prevent out of bounds errors
    - Resize the array to be bigger if needed
    - Check comment for more details

Grading

- I have provided several Geaux programs
  - **<u>Correct</u>** compilation of these programs will result in a guaranteed minimum grade for the coding portion of the grade
  - Grade marked in comments of the program
- 10 minute video this time. Check handout for details